



Programming in C. Recursion.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	0	0	1	2	2	2	1	1	0	2	0
2 Marks Count	0	0	2	2	1	1	0	2	2	2	0
Total Marks	0	0	5	6	4	4	1	5	4	6	0

Welcome to the "Programming and DS: Programming" chapter, specifically focusing on the crucial aspect of **Output**. This subject is fundamental to understanding how programs execute and interact with the user or external systems. For the GATE Computer Science exam, mastering output prediction is not merely about knowing syntax; it's about deeply comprehending the semantics of programming languages, including data types, operators, control flow, memory management, and function calls. Questions related to output prediction are ubiquitous in the GATE CS paper, often integrated into problems involving C/C++/Python code snippets. They typically carry a weightage of 1-2 marks per question, but their underlying concepts are essential for solving larger programming problems. You can expect Multiple Choice Questions (MCQs), Multiple Select Questions (MSQs), and Numerical Answer Type (NAT) questions where you must accurately determine the final output of a given program or code segment.

Topic-wise Key Concepts

Basic Output Functions (C/C++/Python)

This section covers the primary mechanisms for displaying information to the console, which is critical for verifying program logic and understanding execution flow.

- **Definition and Core Idea:** Output functions are library routines or built-in statements that send data from a program to a standard output device, typically the console. They allow programs to communicate results, status, or prompts to the user.

- **Important Formulas, Theorems, and Results:**

- **C - printf():** The standard output function in C.

```
int printf(const char *format, ...);
```

Returns the number of characters printed, or a negative value on error.

- **Format Specifiers:** These tell printf how to interpret and display the corresponding argument.

- %d or %i: Signed decimal integer
- %u: Unsigned decimal integer
- %o: Unsigned octal integer
- %x or %X: Unsigned hexadecimal integer (lowercase/uppercase)
- %f: Decimal floating point (float or double)
- %lf: Used with scanf for double, but %f works for printf for both float and double due to default argument promotions.
- %e or %E: Scientific notation (lowercase/uppercase 'e')
- %g or %G: Shorter of %f or %e
- %c: Character
- %s: String (null-terminated character array)
- %p: Pointer address (implementation-defined format)
- %%%: Prints a literal % character
- %n: Writes the number of characters written so far into the integer pointed to by the corresponding argument.

- **Flags (optional):**

- -: Left-justify
- +: Always print sign for signed numbers
- (space): Print space if no sign
- 0: Pad with leading zeros
- #: Alternate form (e.g., 0x for hex, leading zero for octal, decimal point for floats)

- **Width (optional):** Minimum field width.

- **Precision (optional):** For floats, number of digits after decimal; for strings, max number of characters to print.

- **C++ - cout:** The standard output stream object in C++.

```
std::cout << expression << expression ...;
```

Uses operator overloading for various data types. No format specifiers like printf, but manipulators can be used.

- **Manipulators:**

- `std::endl`: Inserts a newline character and flushes the buffer.
- `std::fixed`: Uses fixed-point notation for floating-point numbers.
- `std::scientific`: Uses scientific notation.
- `std::setprecision(n)`: Sets the decimal precision (total digits for default, digits after decimal for fixed/scientific).
- `std::setw(w)`: Sets field width for the next item.
- `std::left`, `std::right`: Justification.
- `std::hex`, `std::oct`, `std::dec`: Base conversion.

- **Python - print():** The built-in function for output in Python.

`print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`

- `*objects`: One or more objects to be printed.
- `sep=' '`: Separator between objects (default is space).
- `end='\n'`: String appended after the last object (default is newline).
- `file=sys.stdout`: Output stream (default is standard output).

Formatted String Literals (f-strings) (Python 3.6+):

`f"text {variable_name:format_specifier} text"`

Example: `f"Value: {x:.2f}"` for two decimal places.

- **Escape Sequences (Common to C/C++/Python strings):**

- `\n`: Newline
- `\t`: Horizontal tab
- `\b`: Backspace
- `\r`: Carriage return
- `\\`: Backslash
- `\'`: Single quote
- `\"`: Double quote
- `\0`: Null character (string terminator in C/C++)
- `\xhh`: Hexadecimal value hh
- `\ooo`: Octal value ooo

- **Key Properties and Identities:**

- `printf` returns the count of characters printed.
- `cout` returns a reference to `std::cout`, allowing chaining.
- `print()` in Python always returns `None`.
- String literals in C/C++ are null-terminated.
- Floating-point precision issues can lead to unexpected output.

- **Common Pitfalls or Tricky Points:**

- **Mismatching format specifiers in C printf:** Using `%d` for a float or `%f` for an int leads to undefined behavior, often printing garbage.
- **Missing arguments for printf:** If a format specifier is present but no corresponding argument, it can lead to crashes or printing garbage from the stack.
- **Forgetting `std::endl` vs. `\n` in C++:** `std::endl` flushes the buffer, which can be slower. `\n` just inserts a newline.
- **Python `print()` default behavior:** Forgetting `sep` or `end` arguments can lead to unintended spacing or newlines.
- **Precision with floating-point numbers:** Output might be truncated or rounded differently than expected due to internal representation.
- **Buffer flushing:** Output might not appear immediately if the buffer is not flushed (e.g., after `printf` without a newline, or `cout` without `endl`).

- **Standard Problem-Solving Techniques:**

- **Trace format specifiers carefully:** For C `printf`, ensure each format specifier matches the type and order of the arguments.
- **Understand default output behavior:** For C++, know that `cout` automatically handles types. For Python, remember default `sep=' '` and `end='\n'`.
- **Dry run with specific values:** Substitute variables with their values and mentally execute the output statement.
- **Pay attention to escape sequences:** These can drastically alter the visual layout of the output.

Data Types and Type Casting

The way data is stored and interpreted directly influences its output. Type casting allows explicit conversion, which is a common source of tricky output questions.

- **Definition and Core Idea:** Data types define the kind of values a variable can hold and the operations that can be

performed on them. Type casting is the process of converting a value from one data type to another, either implicitly (automatic) or explicitly (programmer-specified).

- **Important Formulas, Theorems, and Results:**

- **Integer Types (C/C++):** char, short, int, long, long long (signed/unsigned variants). Sizes are implementation-dependent but typically:
 - char: 1 byte
 - short: 2 bytes
 - int: 2 or 4 bytes (commonly 4)
 - long: 4 or 8 bytes (commonly 4 on 32-bit, 8 on 64-bit)
 - long long: 8 bytes
- **Floating-Point Types (C/C++):** float, double, long double.
 - float: 4 bytes (single precision)
 - double: 8 bytes (double precision)
 - long double: 10 or 16 bytes
- **Implicit Type Conversion (Promotion):** Occurs automatically in expressions to prevent data loss. Smaller types are promoted to larger types (e.g., int to float, float to double).
Example: int i = 5; float f = 2.5; float result = i + f; (i is promoted to float).
- **Explicit Type Casting (C-style):** (type)expression
Example: int x = (int)3.14; (x becomes 3)
- **Explicit Type Casting (C++-style):**
 - static_cast<type>(expression): For safe and predictable conversions (e.g., numeric conversions).
 - reinterpret_cast<type>(expression): For low-level, potentially unsafe conversions (e.g., pointer to integer).
- **Integer Division:** In C/C++/Python 2, division of two integers results in an integer (truncates fractional part).
Example: int a = 7 / 3; (a becomes 2)

In Python 3, / performs float division, // performs integer (floor) division.

- **Key Properties and Identities:**

- When a larger type is cast to a smaller type, data loss can occur (e.g., double to int truncates, int to char can lose higher-order bits).
- Unsigned integers wrap around on overflow/underflow.
- Character types are often treated as small integers in arithmetic operations (ASCII values).

- **Common Pitfalls or Tricky Points:**

- **Integer division vs. floating-point division:** 5/2 is 2 in C/C++, but 5.0/2 is 2.5.
- **Loss of precision/data:** Casting a large int to a float can lose precision if the int value exceeds the float's representable range. Casting a negative number to an unsigned type can result in a large positive number.
- **Character arithmetic:** 'A' + 1 results in 'B' (assuming ASCII).
- **Order of operations with casting:** (float)a / b vs. (float)(a / b) yield different results if a and b are integers.

- **Standard Problem-Solving Techniques:**

- **Identify implicit conversions:** Trace expressions to see how types are promoted.
- **Evaluate explicit casts carefully:** Understand the potential for truncation or data loss.
- **Pay attention to integer division:** Always check if operands are integers or floats.
- **Consider the range of data types:** Especially for questions involving overflow/underflow or conversions between signed/unsigned types.

Operators and Expressions

Operators combine variables and values to form expressions, whose evaluation order is governed by precedence and associativity, directly impacting the final output.

- **Definition and Core Idea:** Operators are symbols that perform operations on one or more operands. An expression is a combination of operators, operands, and function calls that evaluates to a single value.

- **Important Formulas, Theorems, and Results:**

- **Arithmetic Operators:** +, -, *, /, % (modulus).
 - % operator only works with integer operands.
- **Relational Operators:** == (equal to), != (not equal to), <, >, <=, >=.
 - Return 0 (false) or 1 (true) in C/C++. Return True or False in Python.
- **Logical Operators:**
 - C/C++: && (AND), || (OR), ! (NOT).
 - Short-circuit evaluation: && evaluates right operand only if left is true. || evaluates right operand only if left is false.
 - Python: and, or, not.
- **Bitwise Operators (C/C++):** & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift).

- $x \ll y$ is equivalent to $x \times 2^y$.
- $x \gg y$ is equivalent to $x / 2^y$. For signed numbers, right shift behavior (arithmetic vs. logical) can be implementation-defined for negative numbers.
- **Assignment Operators:** =, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=.
- **Increment/Decrement Operators (C/C++):** ++ (increment), -- (decrement).
 - Prefix (++x): Increments then uses the new value.
 - Postfix (x++): Uses the current value then increments.
- **Conditional (Ternary) Operator (C/C++):** condition ? expression1 : expression2.
 - If condition is true, expression1 is evaluated; otherwise, expression2 is evaluated.
- **Comma Operator (C/C++):** expression1, expression2.
 - Evaluates expression1, discards its result, then evaluates expression2 and returns its result.
- **Operator Precedence and Associativity:** (See Quick Formula Reference for detailed table)
 - Precedence determines which operator is evaluated first in an expression (e.g., * before +).
 - Associativity determines evaluation order for operators of the same precedence (e.g., left-to-right for arithmetic, right-to-left for assignment).
- **Key Properties and Identities:**
 - In C/C++, any non-zero value is considered true in a boolean context; zero is false.
 - Side effects: Operators like ++, --, and assignment operators modify the value of an operand.
 - Undefined behavior: Modifying a variable multiple times within the same expression without an intervening sequence point (e.g., `i = i++ + ++i;`) leads to undefined behavior.
- **Common Pitfalls or Tricky Points:**
 - **= vs. ==:** Accidental assignment in a conditional (e.g., `if (x = 0)` instead of `if (x == 0)`). The assignment evaluates to 0 (false), so the block might not execute.
 - **Operator precedence errors:** Misinterpreting the order of operations can lead to incorrect results (e.g., `1 + 2 * 3` is 7, not 9).
 - **Side effects with increment/decrement:** Understanding when the value is used and when it's updated (prefix vs. postfix) is crucial, especially in complex expressions.
 - **Short-circuiting:** Logical operators && and || can prevent the evaluation of the right-hand operand, which can hide side effects or prevent errors (e.g., null pointer dereference).
 - **Bitwise operations on signed numbers:** Right shift of negative numbers can be tricky due to sign extension.
 - **Comma operator:** Often misunderstood; only the last expression's value is returned.
- **Standard Problem-Solving Techniques:**
 - **Parenthesize ambiguous expressions:** If unsure about precedence, add parentheses.
 - **Trace side effects:** Keep track of variable values *before* and *after* each operation, especially with ++/--.
 - **Evaluate expressions step-by-step:** Apply precedence and associativity rules rigorously.
 - **Understand short-circuiting:** Determine if the right-hand side of a logical expression will be evaluated.

Control Flow Statements

Control flow statements dictate the order in which instructions are executed, determining which parts of the code contribute to the final output.

- **Definition and Core Idea:** Control flow statements manage the sequence of execution of program instructions. They include conditional statements (if-else, switch) for decision-making and loop statements (for, while, do-while) for repetition.
- **Important Formulas, Theorems, and Results:**
 - **if-else (C/C++/Python):**

```
if (condition) { /* statements */ } else { /* statements */ }
```

In Python, if condition: ... elif condition: ... else: ...
 - **switch-case (C/C++):**

```
switch (expression) { case constant1: ... break; case constant2: ... break; default: ... }
```

 - expression must evaluate to an integer type.
 - break is crucial to exit the switch; otherwise, execution "falls through" to the next case.
 - **for loop (C/C++/Python):**

C/C++: `for (initialization; condition; update) { /* statements */ }`

Python: `for item in iterable: /* statements */`
 - **while loop (C/C++/Python):**

```
while (condition) { /* statements */ }
```
 - **do-while loop (C/C++):**

```
do { /* statements */ } while (condition);
```

- Guarantees at least one execution of the loop body.
- **break statement:** Terminates the innermost loop or switch statement immediately.
- **continue statement:** Skips the rest of the current iteration of the innermost loop and proceeds to the next iteration.
- **goto statement (C/C++):** Unconditional jump to a labeled statement. Generally discouraged but can appear in GATE questions.
- **Key Properties and Identities:**
 - Conditions in C/C++ are evaluated as true if non-zero, false if zero.
 - Loop termination conditions are critical.
 - Scope of variables declared within loop headers or blocks.
- **Common Pitfalls or Tricky Points:**
 - **Dangling else:** An else always binds to the nearest preceding unmatched if.
if (cond1) if (cond2) stmt1; else stmt2; (else binds to if (cond2))
 - **Missing break in switch:** Leads to fall-through, executing subsequent case blocks.
 - **Infinite loops:** Incorrect loop conditions or update statements (e.g., while(1) without a break, or for(;;)).
 - **Off-by-one errors in loops:** Incorrect loop bounds (e.g., <= N vs. < N).
 - **Loop variables scope:** A variable declared in a for loop's initialization part is typically scoped to the loop itself (C99, C++, Python).
 - **Side effects in loop conditions:** while (i++ < 10) can be tricky.
- **Standard Problem-Solving Techniques:**
 - **Dry run loop iterations:** Manually trace the values of loop variables and conditions for each iteration.
 - **Identify loop termination:** Determine exactly when a loop will exit.
 - **Check for break and continue:** These alter normal loop flow.
 - **Understand switch fall-through:** Assume fall-through unless a break is present.
 - **Trace nested loops carefully:** Inner loops complete all iterations for each iteration of the outer loop.

Functions

Functions encapsulate reusable code, and their call mechanisms (pass by value/reference) and recursion significantly influence variable states and output.

- **Definition and Core Idea:** A function is a block of organized, reusable code that performs a single, related action. They allow modular programming and abstraction.
- **Important Formulas, Theorems, and Results:**
 - **Call by Value (C/C++/Python):** Arguments are passed as copies. Changes to parameters inside the function do not affect the original arguments in the caller.
Example (C):

```
void func(int x) { x = 10; }
int main() { int a = 5; func(a); printf("%d", a); // Output: 5 }
```
 - **Call by Reference (C/C++ using pointers/references):** Arguments are passed by their memory addresses (pointers) or aliases (references). Changes to parameters inside the function directly affect the original arguments.
Example (C with pointers):

```
void func(int *x) { *x = 10; }
int main() { int a = 5; func(&a); printf("%d", a); // Output: 10 }
```


Example (C++ with references):

```
void func(int &x) { x = 10; }
int main() { int a = 5; func(a); std::cout << a; // Output: 10 }
```
 - **Recursion:** A function calling itself.
 - **Base Case:** A condition that stops the recursion. Without it, infinite recursion occurs.
 - **Recursive Step:** The function calls itself with a modified input, moving towards the base case.
 - **Function Prototypes (C/C++):** Declare a function's return type, name, and parameter types before its definition.
return_type function_name(parameter_type1, parameter_type2, ...);
 - **Return Value:** A function can return a single value using the return statement. If no return is specified for a non-void function, it's undefined behavior in C/C++. Python functions implicitly return None if no return statement is reached.
- **Key Properties and Identities:**

- Local variables within a function have automatic storage duration and are destroyed upon function exit.
- Static local variables retain their value across function calls.
- Global variables are accessible from any function.
- Function call stack: Each function call creates a new stack frame for its local variables and parameters.
- **Common Pitfalls or Tricky Points:**
 - **Forgetting & or * with pointers:** Incorrectly passing by value when reference was intended, or dereferencing issues.
 - **Missing base case in recursion:** Leads to stack overflow (infinite recursion).
 - **Incorrect base case logic:** Can lead to incorrect results or infinite recursion.
 - **Returning address of local variable:** A local variable's memory is deallocated after the function returns, making its address invalid (dangling pointer).
 - **Side effects in function calls:** If a function modifies global variables or uses call-by-reference, it affects the program state outside the function.
 - **Order of evaluation of function arguments:** In C/C++, the order in which function arguments are evaluated is unspecified.
- **Standard Problem-Solving Techniques:**
 - **Trace function calls:** Keep track of the call stack for recursive functions.
 - **Distinguish call by value vs. call by reference:** Determine if the original arguments are modified.
 - **Identify base cases and recursive steps:** Crucial for understanding recursive function behavior.
 - **Track variable scope and lifetime:** Know which variables are accessible and when they exist.
 - **Dry run with small inputs:** Especially for recursive functions, trace a few calls.

Arrays and Pointers

Arrays and pointers are intimately related in C/C++ and are frequent subjects of output questions, often involving memory addresses and dereferencing.

- **Definition and Core Idea:** An array is a collection of elements of the same data type stored in contiguous memory locations. A pointer is a variable that stores the memory address of another variable.
- **Important Formulas, Theorems, and Results:**
 - **Array Declaration (C/C++):** `type array_name[size];`
Example: `int arr[5];`
 - **Array Indexing:** Elements are accessed using `array_name[index]`, where index ranges from 0 to size-1.
 - **Array-Pointer Duality (C/C++):** An array name, when used in an expression (except with `sizeof` or `&`), decays into a pointer to its first element.
`arr` is equivalent to `&arr[0]`.
`arr[i]` is equivalent to `*(arr + i)`.
 - **Pointer Declaration (C/C++):** `type *pointer_name;`
Example: `int *ptr;`
 - **Address-of Operator (C/C++):** `&` returns the memory address of a variable.
 - **Dereference Operator (C/C++):** `*` accesses the value at the memory address stored in a pointer.
 - **Pointer Arithmetic (C/C++):**
 - `ptr + n`: Moves the pointer `n * sizeof(type)` bytes forward.
 - `ptr - n`: Moves the pointer `n * sizeof(type)` bytes backward.
 - `ptr1 - ptr2`: Difference between two pointers of the same type (results in number of elements between them).
 - Increment/Decrement: `ptr++`, `ptr--` move by `sizeof(type)`.
 - **Pointers to Pointers (C/C++):** `type **ptr_to_ptr;`
Example: `int **pp;`
 - **Dynamic Memory Allocation (C):**
 - `malloc(size_t size)`: Allocates `size` bytes. Returns `void*`.
 - `calloc(size_t num, size_t size)`: Allocates `num * size` bytes and initializes to zero.
 - `realloc(void *ptr, size_t new_size)`: Resizes previously allocated memory.
 - `free(void *ptr)`: Deallocates memory.
- **Key Properties and Identities:**
 - Arrays are stored contiguously.
 - Pointers can be assigned, compared, and used in arithmetic.
 - NULL pointer: A pointer that points to no valid memory location (often 0).
 - `sizeof(array_name)` gives the total size of the array in bytes. `sizeof(pointer_name)` gives the size of the pointer variable itself (e.g., 4 or 8 bytes).
- **Common Pitfalls or Tricky Points:**
 - **Array out-of-bounds access:** Accessing `arr[size]` or negative indices leads to undefined behavior

(reading/writing to arbitrary memory).

- **Dangling pointers:** Pointers that point to deallocated memory.
- **Wild pointers:** Uninitialized pointers that point to arbitrary memory.
- **Pointer type mismatch:** Assigning a pointer of one type to another without explicit casting can lead to incorrect dereferencing.
- **Confusion between array name and pointer:** While `arr` decays to `&arr[0]`, `sizeof(arr)` is different from `sizeof(&arr[0])`.
- **Incorrect pointer arithmetic:** Adding an integer to a pointer moves it by multiples of the pointed-to type's size.
- **Memory leaks:** Forgetting to free dynamically allocated memory.
- **Standard Problem-Solving Techniques:**
 - **Draw memory diagrams:** Visualize memory addresses, array elements, and what pointers point to.
 - **Trace pointer values:** Keep track of the address stored in each pointer variable.
 - **Trace dereferenced values:** When `*ptr` is used, find the value at that address.
 - **Calculate pointer arithmetic carefully:** Remember to multiply by `sizeof(type)`.
 - **Check array bounds:** Ensure all array accesses are within 0 to `size-1`.

Strings

Strings, often implemented as character arrays in C/C++, involve specific functions and memory handling that are frequently tested.

- **Definition and Core Idea:** A string is a sequence of characters. In C/C++, strings are typically null-terminated character arrays. Python has a built-in immutable string type.
- **Important Formulas, Theorems, and Results (C/C++):**
 - **Declaration:** `char str[] = "hello";` or `char *str = "hello";` (read-only string literal).
 - **Null Termination:** All C-style strings end with a null character (`\0`).
 - **Standard Library Functions (from `<string.h>`):**
 - `size_t strlen(const char *s);` Returns the length of string `s` (excluding `\0`).
 - `char *strcpy(char *dest, const char *src);` Copies `src` to `dest`. Returns `dest`.
 - `char *strncpy(char *dest, const char *src, size_t n);` Copies at most `n` characters. Does not guarantee null termination if `src` is longer than `n`.
 - `char *strcat(char *dest, const char *src);` Appends `src` to `dest`. Returns `dest`.
 - `char *strncat(char *dest, const char *src, size_t n);` Appends at most `n` characters.
 - `int strcmp(const char *s1, const char *s2);` Compares `s1` and `s2` lexicographically. Returns `<0` if `s1 < s2`, `0` if `s1 == s2`, `>0` if `s1 > s2`.
 - `int strncmp(const char *s1, const char *s2, size_t n);` Compares at most `n` characters.
 - `char *strstr(const char *haystack, const char *needle);` Finds the first occurrence of `needle` in `haystack`. Returns pointer to first occurrence or `NULL`.
 - `char *strchr(const char *s, int c);` Finds the first occurrence of character `c` in string `s`.
- **Key Properties and Identities:**
 - String literals are often stored in read-only memory. Modifying them leads to undefined behavior.
 - `strcpy` and `strcat` do not perform bounds checking, making them vulnerable to buffer overflows.
 - String comparison functions return integer values based on lexicographical order.
- **Common Pitfalls or Tricky Points:**
 - **Buffer Overflow:** Copying a string larger than the destination buffer using `strcpy` or `strcat`.
 - **Missing Null Terminator:** If a string is not null-terminated, functions like `strlen` or `printf("%s")` will read past the allocated memory, leading to crashes or garbage output. `strncpy` is particularly prone to this if `n` is less than the source string's length.
 - **Modifying String Literals:** Attempting to change characters in `char *s = "hello";` results in a runtime error or undefined behavior. Use `char s[] = "hello";` for modifiable strings.
 - **Incorrect comparison:** Using `==` to compare C-style strings compares their memory addresses, not their content. Use `strcmp()`.
 - **Empty strings:** An empty string `""` contains only a null terminator. `strlen("")` is `0`.
- **Standard Problem-Solving Techniques:**
 - **Trace memory for string operations:** Visualize how characters are copied, concatenated, or modified in memory, including the null terminator.
 - **Check buffer sizes:** Ensure destination buffers are large enough to accommodate source strings plus the null terminator.
 - **Use `strcmp` for content comparison:** Never use `==` for C-style strings.
 - **Be aware of `strncpy` behavior:** It might not null-terminate if the source is too long.

Storage Classes

Storage classes determine the scope, lifetime, and linkage of variables, which directly impacts their values and accessibility at different points in a program's execution.

- **Definition and Core Idea:** Storage classes in C/C++ specify the scope (visibility), lifetime (duration), and linkage (how names are shared across files) of variables and functions.
- **Important Formulas, Theorems, and Results:**
 - **auto (Automatic):**
 - **Scope:** Local to the block in which it's declared.
 - **Lifetime:** Created on entry to the block, destroyed on exit.
 - **Initialization:** Contains garbage value if not explicitly initialized.
 - **Linkage:** No linkage.
 - Default for local variables.
 - **static:**
 - **Local static variables:**
 - **Scope:** Local to the block.
 - **Lifetime:** Persists throughout the program's execution (initialized once).
 - **Initialization:** Zero-initialized by default if not explicitly initialized.
 - **Linkage:** No linkage.
 - **Global static variables/functions:**
 - **Scope:** File scope.
 - **Lifetime:** Program execution.
 - **Initialization:** Zero-initialized by default.
 - **Linkage:** Internal linkage (visible only within the file where declared).
 - **extern (External):**
 - **Scope:** Global (can be accessed from other files).
 - **Lifetime:** Program execution.
 - **Initialization:** Zero-initialized by default.
 - **Linkage:** External linkage (can be shared across multiple files).
 - Used to declare a variable/function defined in another file.
 - **register:**
 - **Scope:** Local to the block.
 - **Lifetime:** Block execution.
 - **Initialization:** Garbage value.
 - **Linkage:** No linkage.
 - Hint to the compiler to store the variable in a CPU register for faster access. Compiler may ignore this hint. Cannot take address of a register variable.
- **Key Properties and Identities:**
 - static variables are initialized only once, even if the function is called multiple times.
 - Global variables (declared outside any function) have extern linkage by default.
 - Uninitialized auto variables have indeterminate values; uninitialized static/extern variables are zero-initialized.
- **Common Pitfalls or Tricky Points:**
 - **Uninitialized auto variables:** Using their values before initialization leads to garbage output or undefined behavior.
 - **Misunderstanding static local variables:** Forgetting they retain their value across function calls.
 - **Scope confusion:** Shadowing a global variable with a local variable of the same name.
 - **extern without definition:** Declaring a variable extern but never defining it in any compilation unit will lead to a linker error.
 - **Taking address of register variable:** This is an error.
- **Standard Problem-Solving Techniques:**
 - **Track variable lifetime:** For each variable, determine when it comes into existence and when it's destroyed.
 - **Identify variable scope:** Determine where each variable is accessible.
 - **Pay attention to initialization:** Differentiate between default zero-initialization and indeterminate values.
 - **Trace function calls with static variables:** Keep a separate record of static variable values that persist.

Macros and Preprocessor Directives

Preprocessor directives modify the source code before compilation, which can subtly alter program behavior and output.

- **Definition and Core Idea:** Preprocessor directives are instructions to the C/C++ preprocessor to perform text substitutions or conditional compilation before the actual compilation phase.
- **Important Formulas, Theorems, and Results:**

- **#define (Macro Definition):**
 - **Object-like macro:** #define PI 3.14159 (simple text replacement).
 - **Function-like macro:** #define MAX(a, b) ((a) > (b) ? (a) : (b)) (takes arguments).
- **#include:** Inserts the content of a specified file.
 - #include <filename>: Searches standard library paths.
 - #include "filename": Searches current directory first, then standard paths.
- **Conditional Compilation:**
 - #ifdef MACRO / #ifndef MACRO: Checks if MACRO is defined/not defined.
 - #if constant_expression: Evaluates a constant integer expression.
 - #elif constant_expression / #else / #endif.
 - defined(MACRO): Operator used within #if to check if a macro is defined.
- **#undef:** Undefines a previously defined macro.
- **#pragma:** Provides compiler-specific directives.
- **Key Properties and Identities:**
 - Macros are simple text substitutions; they do not respect scope or type checking.
 - Function-like macros are expanded inline, potentially leading to faster execution but also larger code size.
 - Preprocessor directives are processed before compilation.
- **Common Pitfalls or Tricky Points:**
 - **Macro argument side effects:** If a macro argument has a side effect (e.g., MAX(i++, j)), it might be evaluated multiple times, leading to unexpected results.
Example: #define SQUARE(x) (x * x). SQUARE(a+b) expands to (a+b * a+b), which is (a + (b*a) + b) due to operator precedence.
 - **Missing parentheses in macros:** Crucial for correct evaluation, especially in function-like macros. Always parenthesize arguments and the entire macro definition.
Correct: #define SQUARE(x) ((x) * (x))
 - **Semicolons after macros:** Don't put a semicolon after a macro definition unless it's part of the replacement text.
 - **Conditional compilation logic:** Incorrect #ifdef/#ifndef or #if conditions can include/exclude wrong code blocks.
- **Standard Problem-Solving Techniques:**
 - **Mentally expand macros:** Replace macro calls with their expanded text before evaluating the expression.
 - **Trace conditional compilation:** Determine which code blocks are included based on macro definitions.
 - **Watch for side effects in macro arguments:** Be extra careful when arguments involve ++, --, or function calls.

Quick Formula Reference

This section provides a consolidated list of essential formulas and rules for quick revision.

C/C++ Operator Precedence and Associativity (Highest to Lowest)

Precedence	Operators	Associativity
1 (Highest)	() [] . -> ::	Left-to-Right
2	++ -- (postfix)	Left-to-Right
3	++ -- (prefix) + - (unary) ! ~ * (dereference) & (address-of) sizeof (type)	Right-to-Left
4	* / %	Left-to-Right
5	+ - (binary)	Left-to-Right
6	<< >>	Left-to-Right
7	< <= > >=	Left-to-Right
8	== !=	Left-to-Right
9	& (bitwise AND)	Left-to-Right
10	^ (bitwise XOR)	Left-to-Right
11	(bitwise OR)	Left-to-Right
12	&& (logical AND)	Left-to-Right
13	(logical OR)	Left-to-Right
14	?: (conditional)	Right-to-Left
15	= += -= *= /= %= &= = ^= <<= >>=	Right-to-Left
16 (Lowest)	, (comma)	Left-to-Right

C printf() Format Specifiers

- %d, %i: signed int
- %u: unsigned int
- %o: unsigned octal
- %x, %X: unsigned hexadecimal
- %f: float/double (decimal)
- %e, %E: float/double (scientific)
- %g, %G: float/double (shorter of %f or %e)
- %c: char
- %s: string (char array)
- %p: pointer address
- %%: literal '%'
- %n: writes count of characters printed so far

Escape Sequences

- \n: Newline
- \t: Tab
- \b: Backspace
- \r: Carriage return
- \\: Backslash
- \' : Single quote
- \": Double quote
- \0: Null character

Pointer Arithmetic (C/C++)

- ptr + n: Address of ptr + n * sizeof(*ptr)
- ptr - n: Address of ptr - n * sizeof(*ptr)
- ptr1 - ptr2: (address_of_ptr1 - address_of_ptr2) / sizeof(*ptr1)

C String Functions (from <string.h>)

- strlen(s): Length of string s (excluding '\0').
- strcpy(dest, src): Copies src to dest.
- strcat(dest, src): Appends src to dest.
- strcmp(s1, s2): Compares s1 and s2. Returns 0 if equal.

Storage Class Properties (C/C++)

- **auto**: Local scope, block lifetime, garbage init, no linkage.
- **static (local)**: Local scope, program lifetime, zero init, no linkage.
- **static (global)**: File scope, program lifetime, zero init, internal linkage.
- **extern**: Global scope, program lifetime, zero init, external linkage.
- **register**: Local scope, block lifetime, garbage init, no linkage (cannot take address).

Important Tips for GATE

1. **Meticulous Dry Running**: For every code snippet, mentally execute it line by line. Keep a scratchpad to track variable values, memory addresses (for pointers/arrays), and the exact output generated at each printf/cout/print() call.
2. **Master Operator Precedence and Associativity**: This is arguably the most common source of errors in output prediction. Memorize the table or at least the relative order of common operators (arithmetic, logical, bitwise, assignment, increment/decrement). When in doubt, use parentheses.
3. **Watch for Side Effects and Sequence Points**: Be extremely cautious with expressions involving increment/decrement operators (++ , --) or multiple assignments within a single statement (e.g., a = b++ + ++c;). If a variable is modified more than once without an intervening sequence point, the behavior is undefined, though GATE questions usually imply a specific, predictable (though tricky) outcome.
4. **Understand Data Types and Type Casting**: Pay close attention to integer division (truncation), implicit type promotions, and explicit type casts. Conversions between signed and unsigned integers, or between integer and floating-point types, can drastically change values.
5. **Trace Control Flow Rigorously**: For loops, identify the initialization, condition, and update parts. Determine the

exact number of iterations. For if-else and switch-case, carefully evaluate conditions and watch for break statements (or their absence, leading to fall-through).

6. **Demystify Pointers and Arrays:** Draw memory diagrams to visualize array elements and what pointers are pointing to. Remember array-pointer duality, pointer arithmetic (sizeof the pointed-to type is crucial), and the difference between `arr` and `&arr[0]`. Be wary of out-of-bounds access.
7. **Grasp Function Call Mechanisms:** Clearly distinguish between call-by-value (copies are passed, originals unaffected) and call-by-reference (pointers/references, originals can be modified). For recursion, identify the base case and the recursive step, and trace a few calls to understand the pattern.
8. **Preprocessor Directives and Macros:** Mentally expand macros before evaluating the code. Be aware of common macro pitfalls like missing parentheses around arguments or the macro body, which can lead to unexpected operator precedence issues.

4.1

Output (1)

4.1.1 Output: GATE CSE 2024 | Set 2 | Question: 23



Consider the following C function definition.

```
int fX(char *a) {  
    char *b = a;  
    while (*b)  
        b++;  
    return b - a; }
```

Which of the following statements is/are TRUE?

- A. The function call `fX("abcd")` will always return a value
- B. Assuming a character array `c` is declared as `char c[] = "abcd"` in `main()`, the function call `fX(c)` will always return a value
- C. The code of the function will not compile
- D. Assuming a character pointer `c` is declared as `char *c = "abcd"` in `main()`, the function call `fX(c)` will always return a value

gatecse-2024-set2 programming programming-in-c multiple-selects output one-mark

Answer key

Answer Keys

4.1.1

A;B;D